Course Notes

Lecture 11 — Session-Layer Protocols for IoT: HTTP, CoAP and XMPP

Communication Technologies for the Internet of Things

Academic year 2026–2027

Catarina Silva

ESTGA, University of Aveiro

Text to support independent study

Contents

How to Use These Notes **2**

1 Commentary on the Slides **2**
2. Agenda (1/2) . . . 2
3. Agenda (2/2) . . . 3
4. Place in the Communication Stack (1/2) . . . 4
5. Place in the Communication Stack (2/2) . . . 5
6. The Protocols in the Stack . . . 6
7. HTTP: Model and Characteristics (1/2) . . . 7
8. HTTP: Model and Characteristics (2/2) . . . 8
9. An HTTP Request over TCP . . . 9
10. Why HTTP Is Unsuitable for Constrained Devices (1/2) . . . 10
11. Why HTTP Is Unsuitable for Constrained Devices (2/2) . . . 11
12. The Cost of Headers . . . 12
13. CoAP: Constrained Application Protocol (1/2) . . . 13
14. CoAP: Constrained Application Protocol (2/2) . . . 14
15. CoAP: Features (1/2) . . . 15
16. CoAP: Features (2/2) . . . 16
17. The CoAP Message . . . 17
18. CoAP: Reliability over UDP (1/2) . . . 18
19. CoAP: Reliability over UDP (2/2) . . . 19
20. Confirmable and Non-Confirmable . . . 20
21. CoAP: Signalling Reduction Compared with HTTP (1/2) . . . 21
22. CoAP: Signalling Reduction Compared with HTTP (2/2) . . . 22
23. XMPP: Extensible Messaging and Presence Protocol (1/2) . . . 23
24. XMPP: Extensible Messaging and Presence Protocol (2/2) . . . 24
25. XMPP in an IoT Context (1/2) . . . 25
26. XMPP in an IoT Context (2/2) . . . 26
27. XMPP Federation . . . 27
28. Comparing the Three Protocols (1/2) . . . 28
29. Comparing the Three Protocols (2/2) . . . 29
30. The Three Protocols in a Table . . . 30
31. A Complete Architecture . . . 31
32. CoAP Implementations (1/2) . . . 32
33. CoAP Implementations (2/2) . . . 33
34. Lecture Summary (1/2) . . . 34
35. Lecture Summary (2/2) . . . 35

Final Check **36**

References **36**

How to Use These Notes

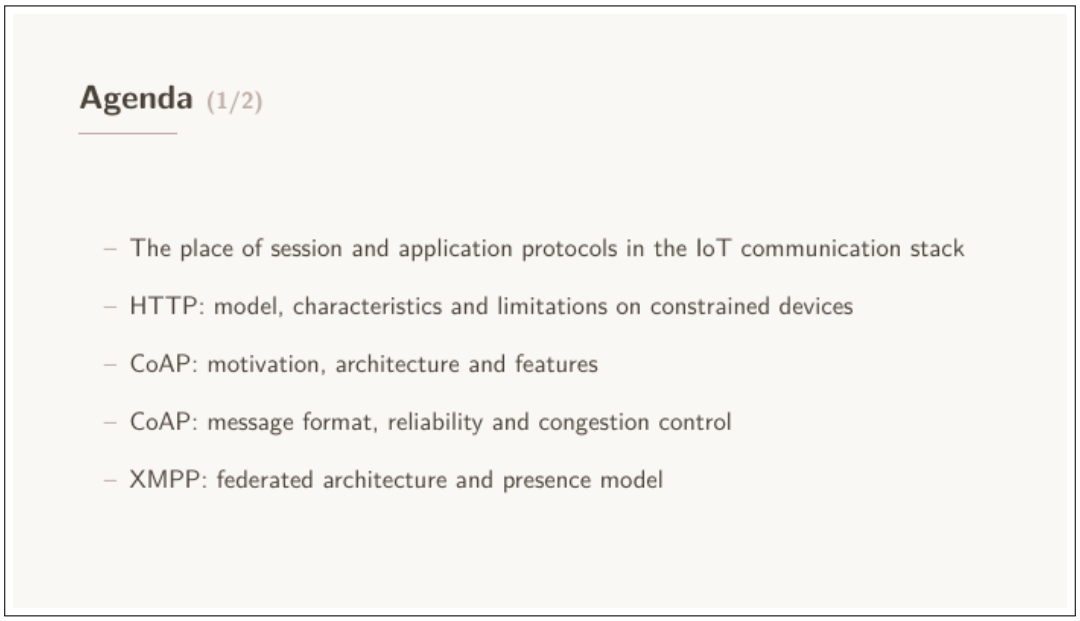
These notes follow the lecture slide by slide. Each page reproduces a slide and states, below it, what should be taken from it.

Reading them does not replace the lecture or the slides. They are meant for later study, once the student has heard the lecture and wants to consolidate, check that the essentials were understood or prepare for assessment. The comments make explicit links between slides that remain implicit in the oral presentation, and point out where the material is commonly misunderstood.

Whenever a comment poses a question, that question is the verification criterion: if you cannot answer it, go back to the slide before moving on.

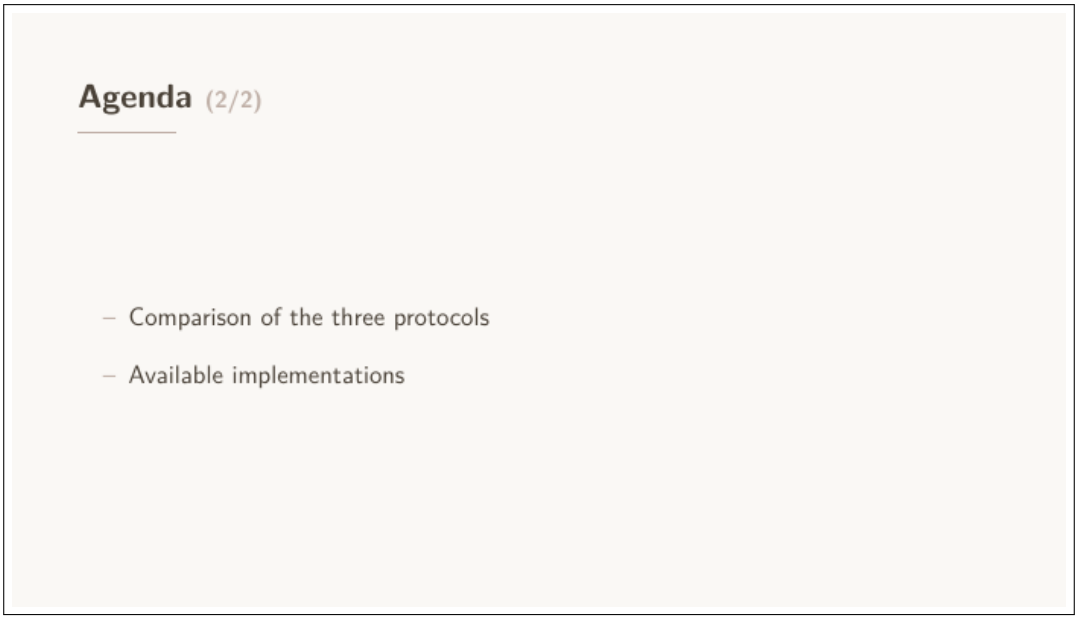
1. Commentary on the Slides

Slide 2. Agenda (1/2)



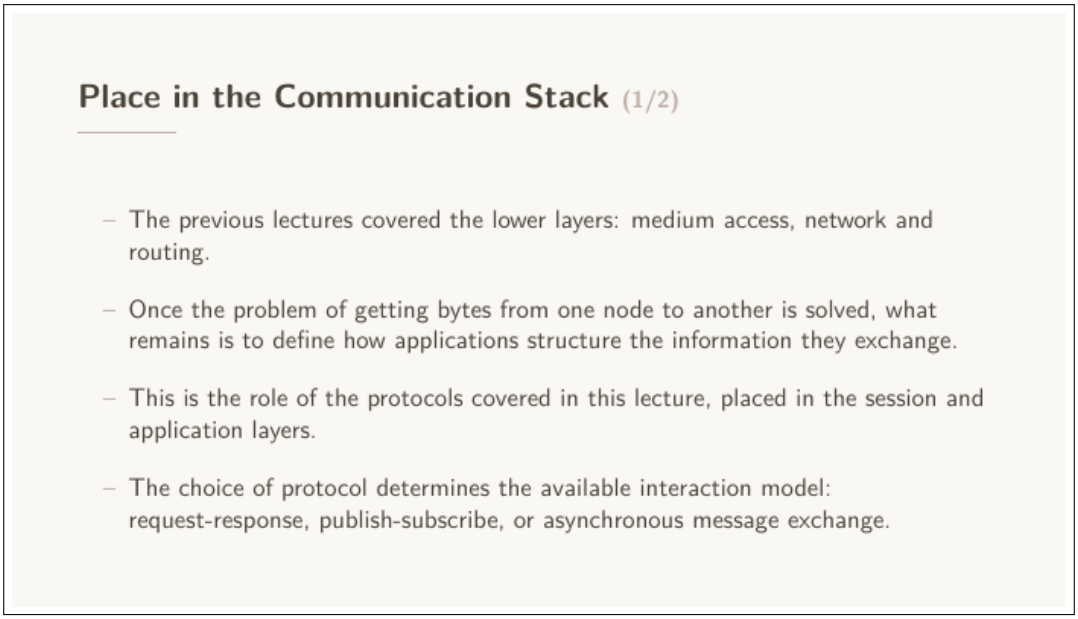
The path through the lecture. The lecture compares three protocols that solve the same problem in different ways: HTTP, CoAP and XMPP. For each, what matters is the interaction model, the cost and the suitability for constrained devices.

Slide 3. Agenda (2/2)



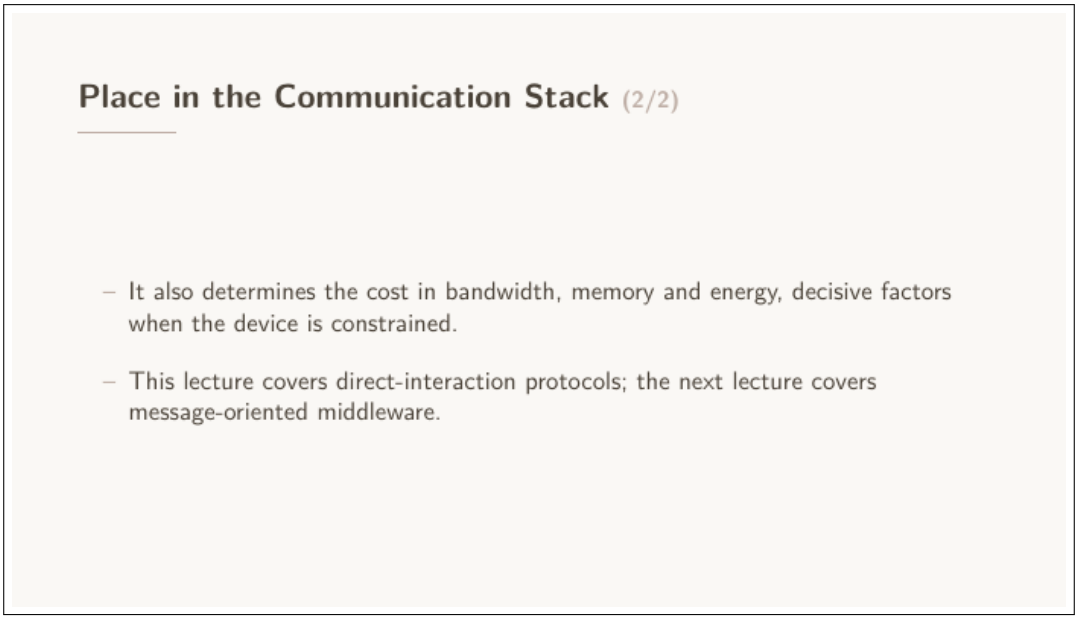
Comparing and choosing. The final comparison and the available implementations serve as a decision guide. Choosing a protocol is choosing a trade-off, and the lecture aims for the student to be able to justify it.

Slide 4. Place in the Communication Stack (1/2)



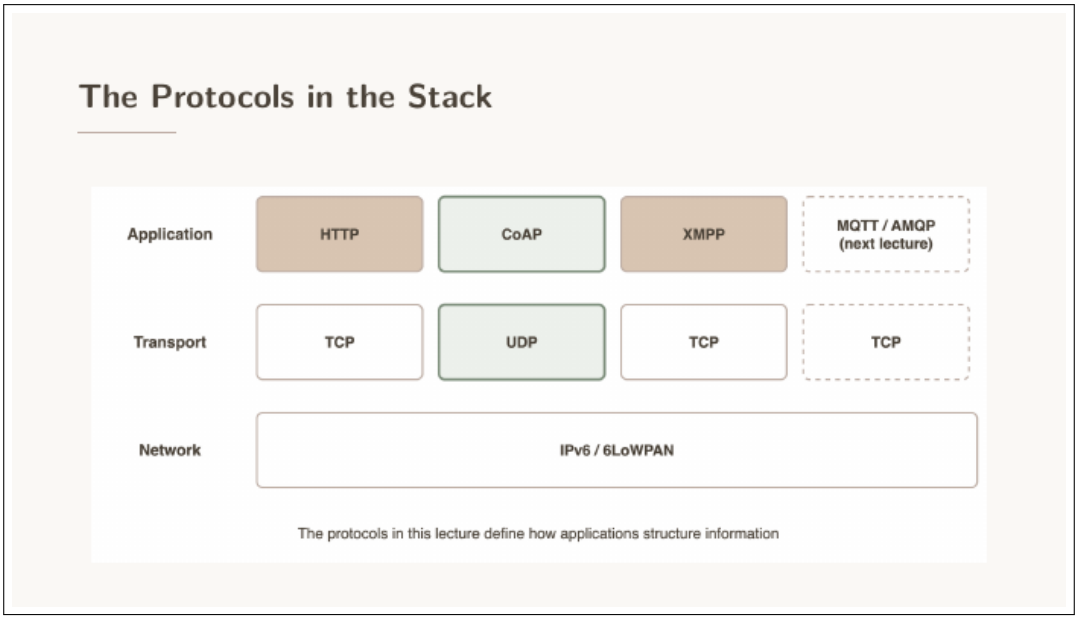
From the lower layers to the application. Once the transport of bytes is solved, what remains is to define how applications structure information. The protocol determines the available interaction model and, with it, the cost in bandwidth, memory and energy.

Slide 5. Place in the Communication Stack (2/2)



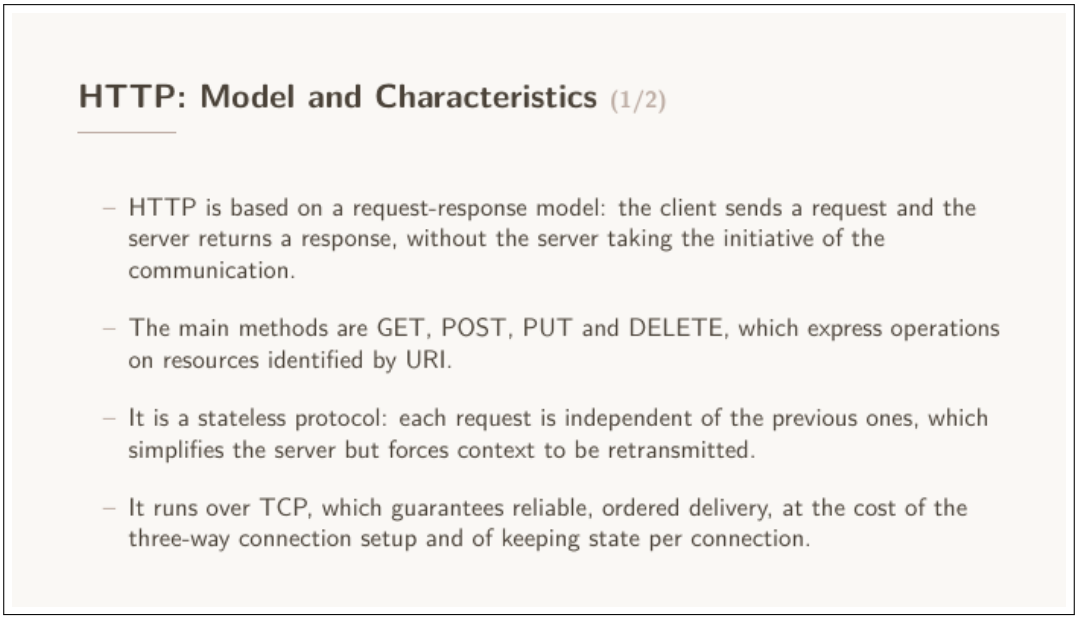
Direct interaction, then middleware. This lecture covers direct-interaction protocols, in which client and server talk to each other. The next lecture covers message-oriented middleware, in which an intermediary decouples producers and consumers.

Slide 6. The Protocols in the Stack



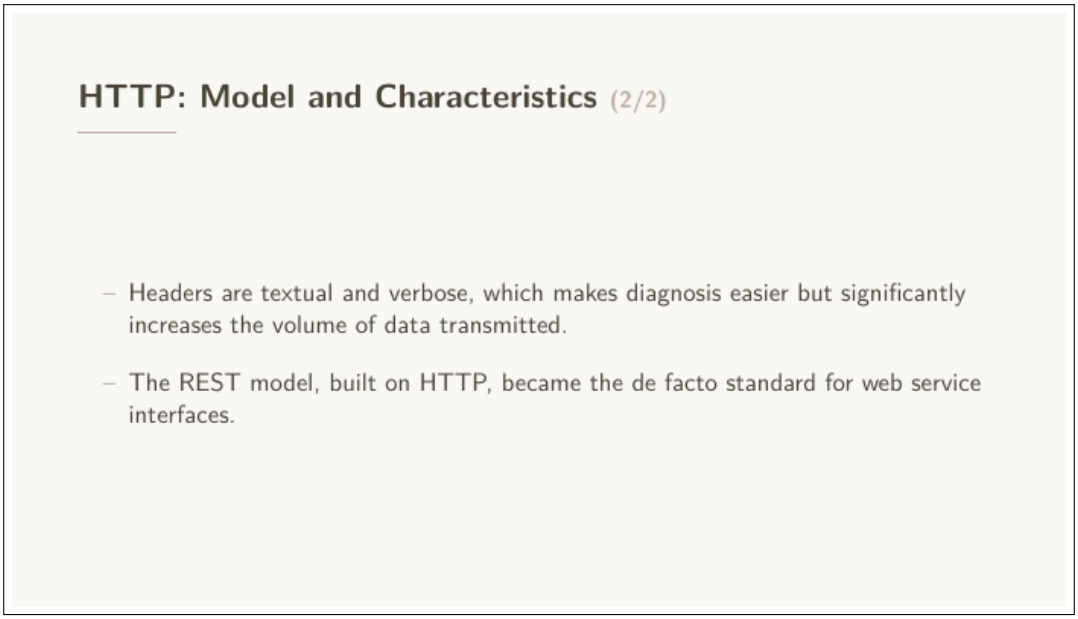
The protocols in the stack. The figure places HTTP, CoAP and XMPP in the application layer and shows the transport of each: TCP for HTTP and XMPP, UDP for CoAP. The dashed box is the subject of the next lecture.

Slide 7. HTTP: Model and Characteristics (1/2)



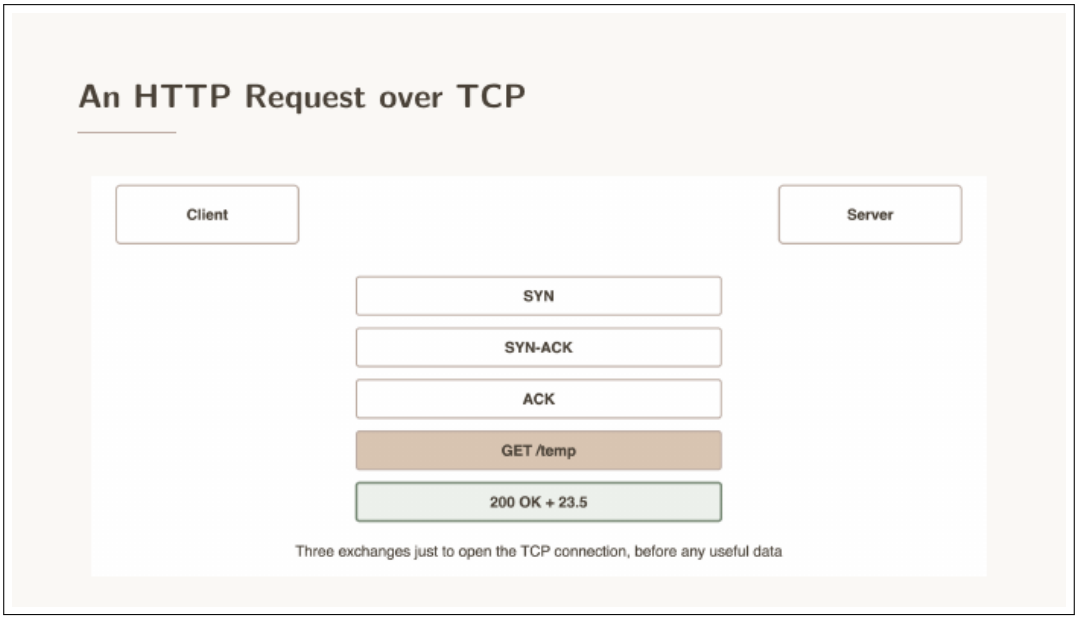
The HTTP model. Request-response, stateless, over TCP. Each of these traits is reasonable on the Web and has a cost on a constrained device: statelessness forces context to be repeated and TCP forces connections to be maintained.

Slide 8. HTTP: Model and Characteristics (2/2)



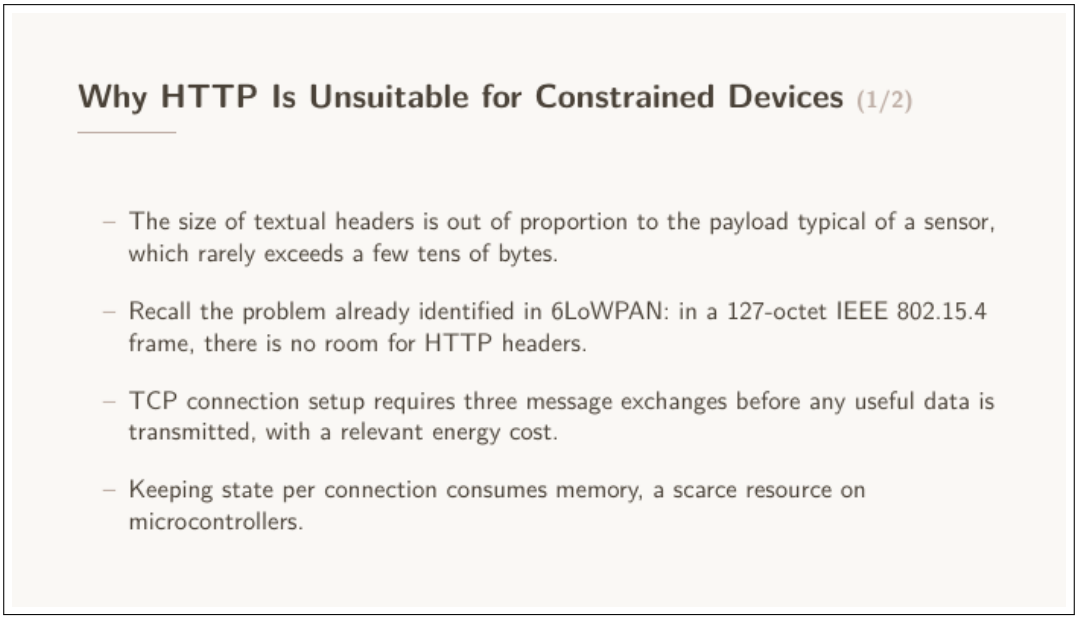
Text and REST. Textual headers help diagnosis and cost octets. REST, built on HTTP, became the standard for service interfaces, and it is this familiarity that CoAP wants to preserve.

Slide 9. An HTTP Request over TCP



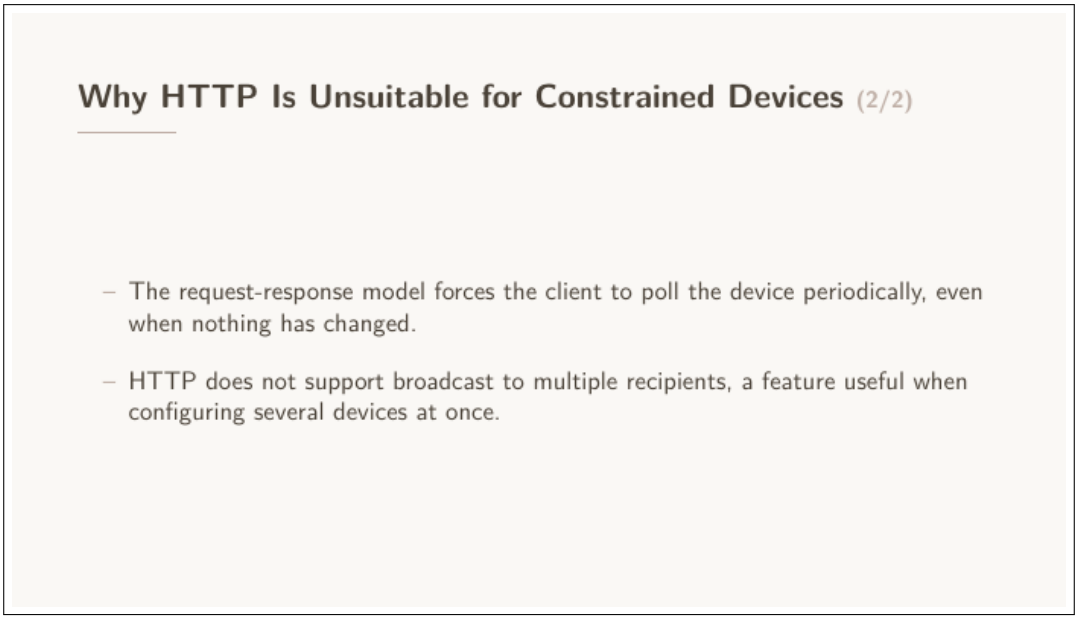
A request in detail. The figure shows what precedes useful data: three exchanges to open the TCP connection. Only then do the GET request and the response follow. Check: how many messages are exchanged until the client receives the value?

Slide 10. Why HTTP Is Unsuitable for Constrained Devices (1/2)



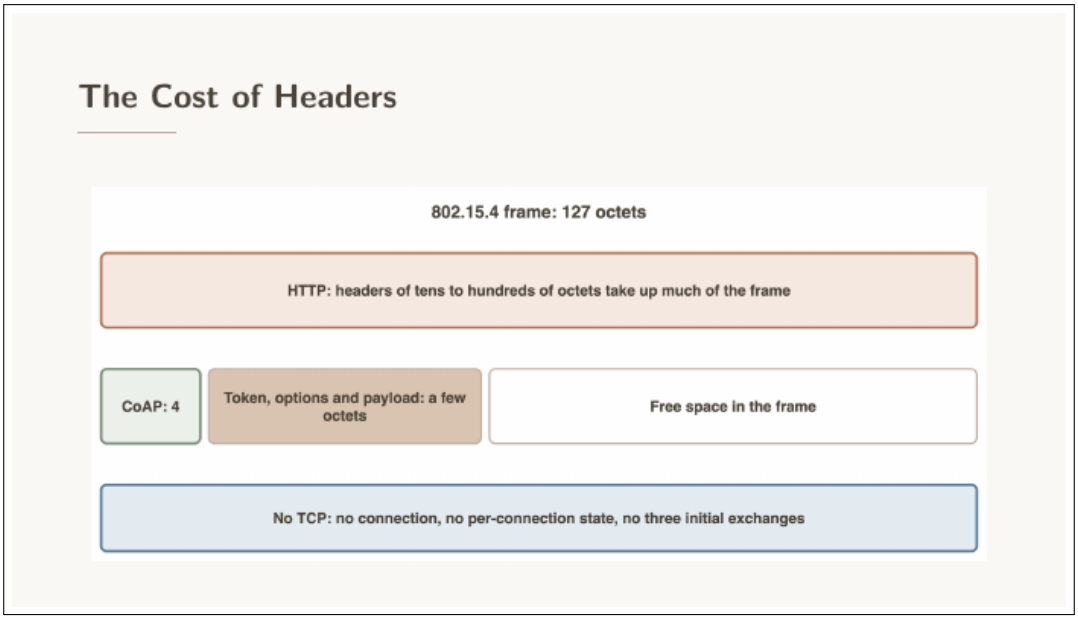
Disproportionate headers. A sensor sends a few bytes of data, and the header of an HTTP request is much larger. In a 127-octet frame the problem is even more evident, as seen with 6LoWPAN.

Slide 11. Why HTTP Is Unsuitable for Constrained Devices (2/2)



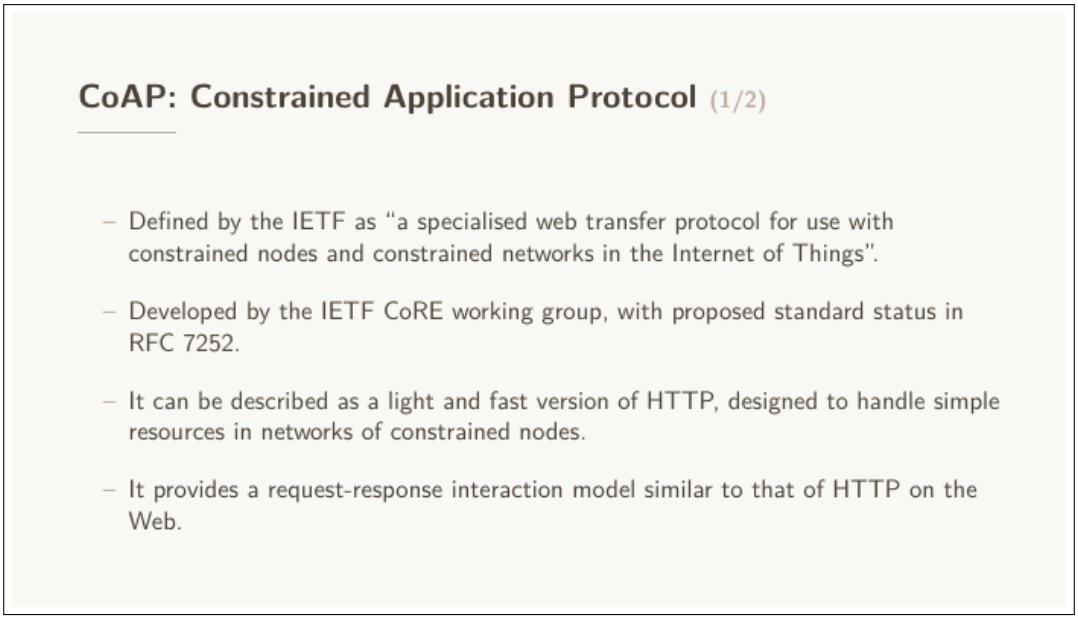
State, polling and multicast. State per connection consumes memory, request-response forces periodic polling and HTTP does not support broadcast. Each limitation weighs more on a microcontroller than on a server.

Slide 12. The Cost of Headers



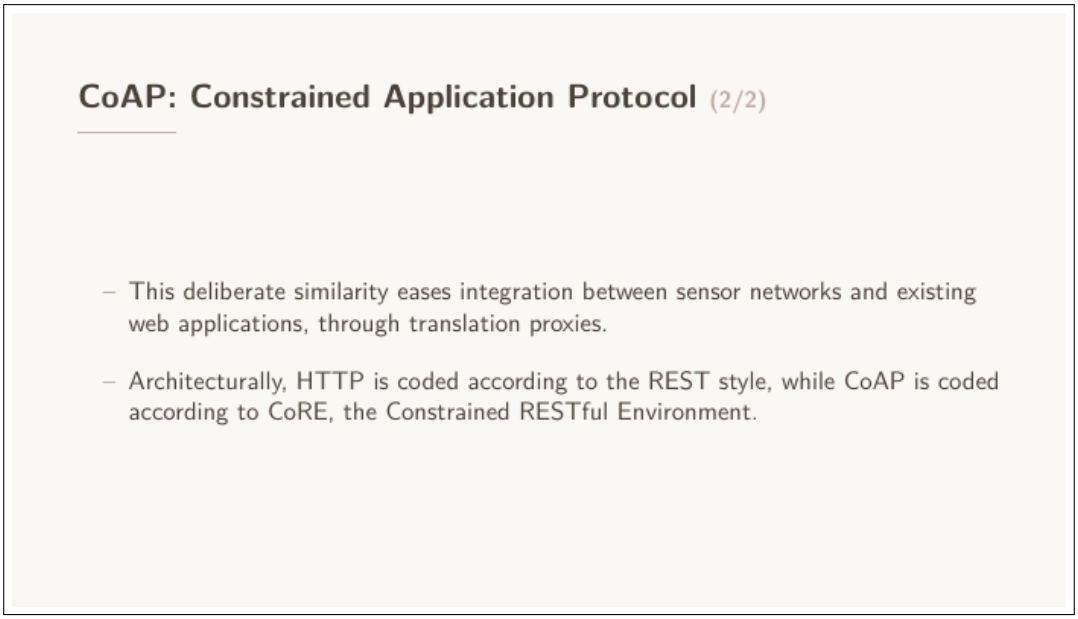
The cost of headers in a picture. The figure contrasts the HTTP header with CoAP's 4-octet base header, and recalls that UDP dispenses with the connection and the state associated with it.

Slide 13. CoAP: Constrained Application Protocol (1/2)



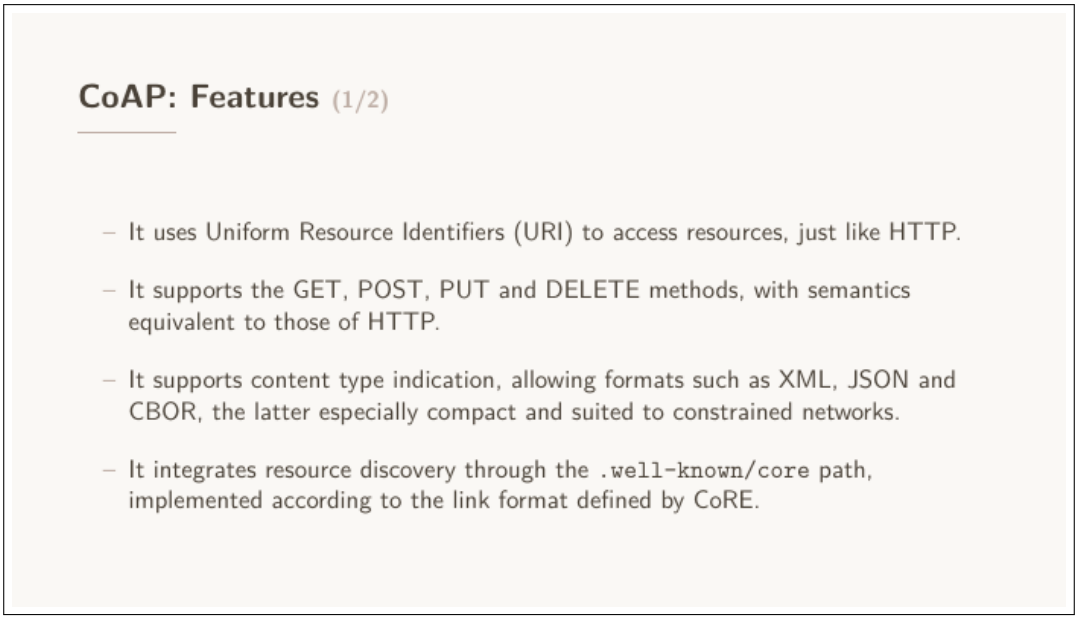
What CoAP is. CoAP is a specialised web transfer protocol, standardised in RFC 7252 by the CoRE group. The idea is to keep the request-response model of HTTP, but adapt it to constrained nodes and networks.

Slide 14. CoAP: Constrained Application Protocol (2/2)



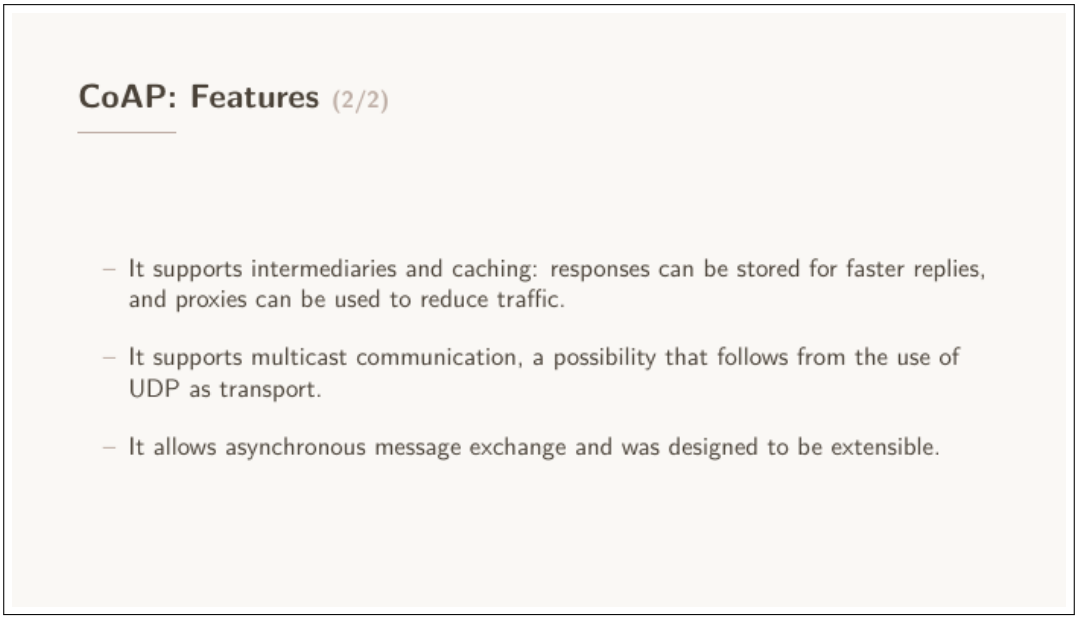
Integrating with the Web. The similarity with HTTP is deliberate: it allows simple bridges, through translation proxies, between the sensor network and existing web applications.

Slide 15. CoAP: Features (1/2)



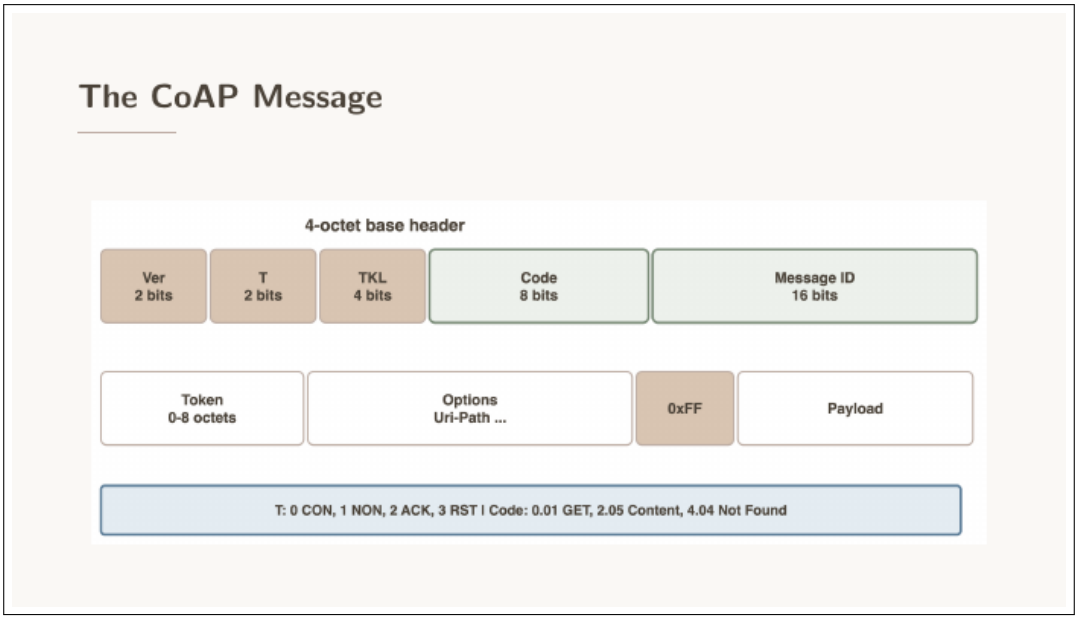
Inherited and new features. URIs, the four methods and content types are inherited from HTTP. CBOR is a compact binary format. Discovery at .well-known/core is new: it lets a client ask which resources a device offers.

Slide 16. CoAP: Features (2/2)



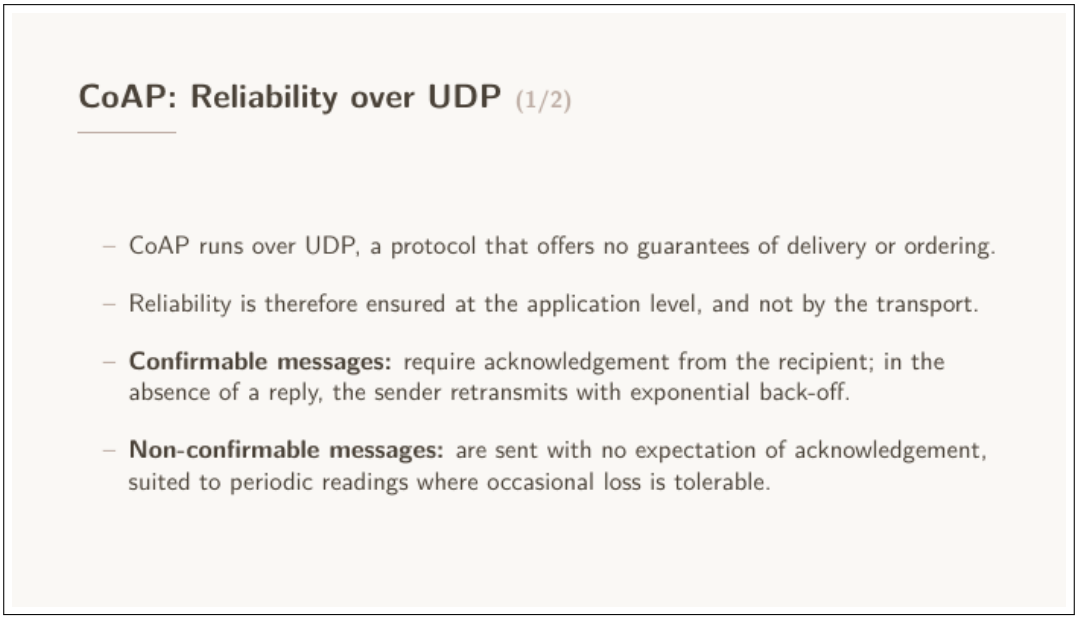
Cache, multicast and asynchronous. Responses can be cached, multicast comes from UDP, and asynchronous exchange lets the server reply later. These features do not exist in classic HTTP.

Slide 17. The CoAP Message



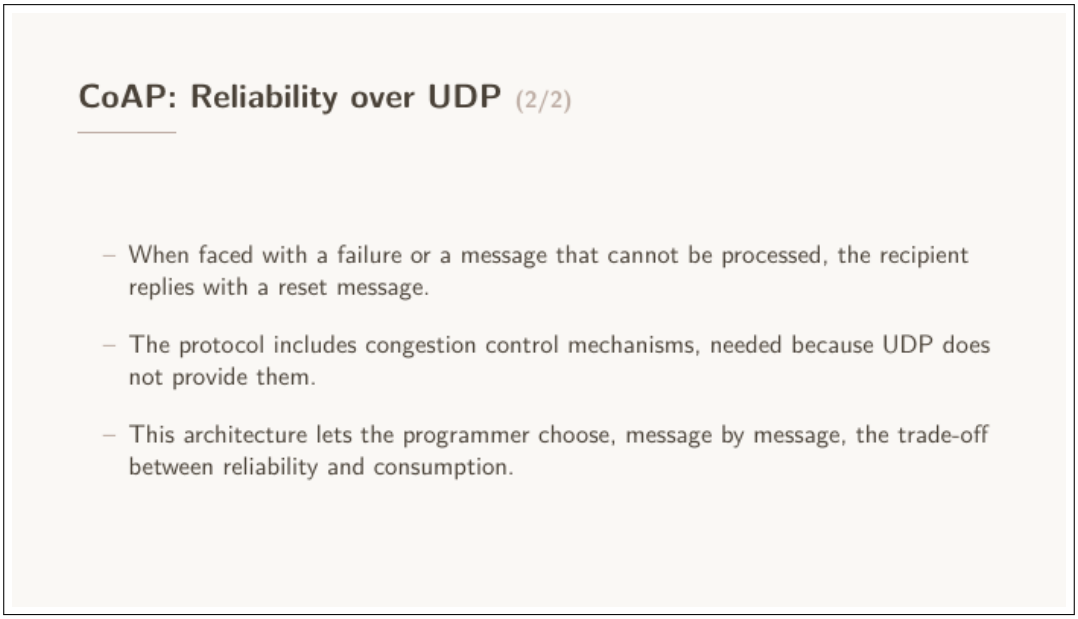
The CoAP message. The base header has 4 octets: version, type, token length, code and message identifier. The token, the options, the 0xFF marker and the payload follow. Code 0.01 is GET and 2.05 is Content.

Slide 18. CoAP: Reliability over UDP (1/2)



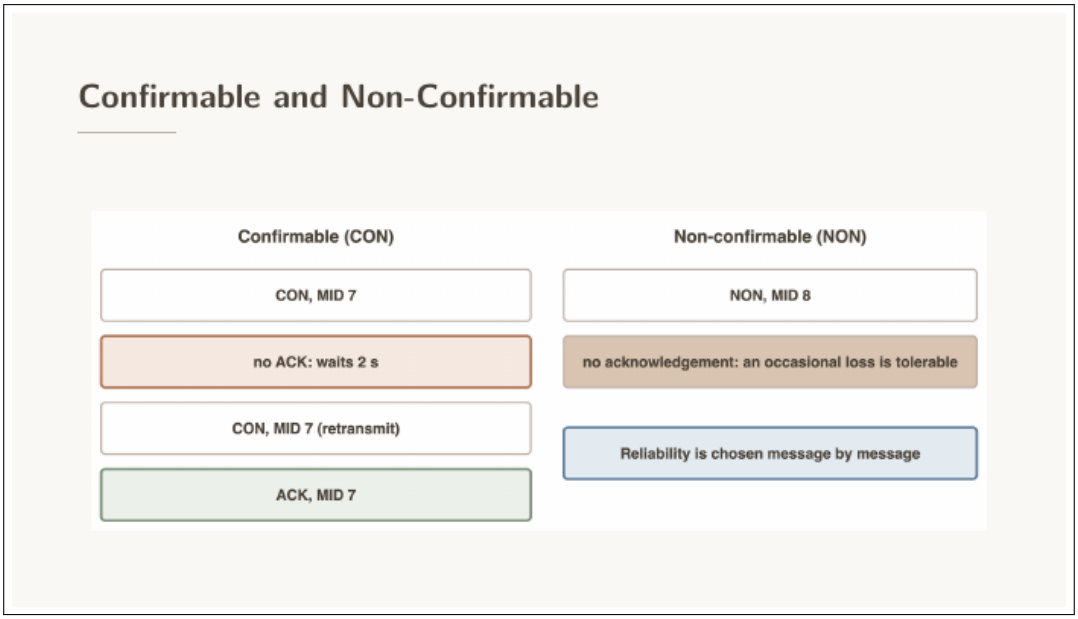
Reliability in the application. UDP guarantees neither delivery nor ordering, so CoAP handles reliability itself. Confirmable messages are retransmitted with exponential back-off until they receive an ACK.

Slide 19. CoAP: Reliability over UDP (2/2)



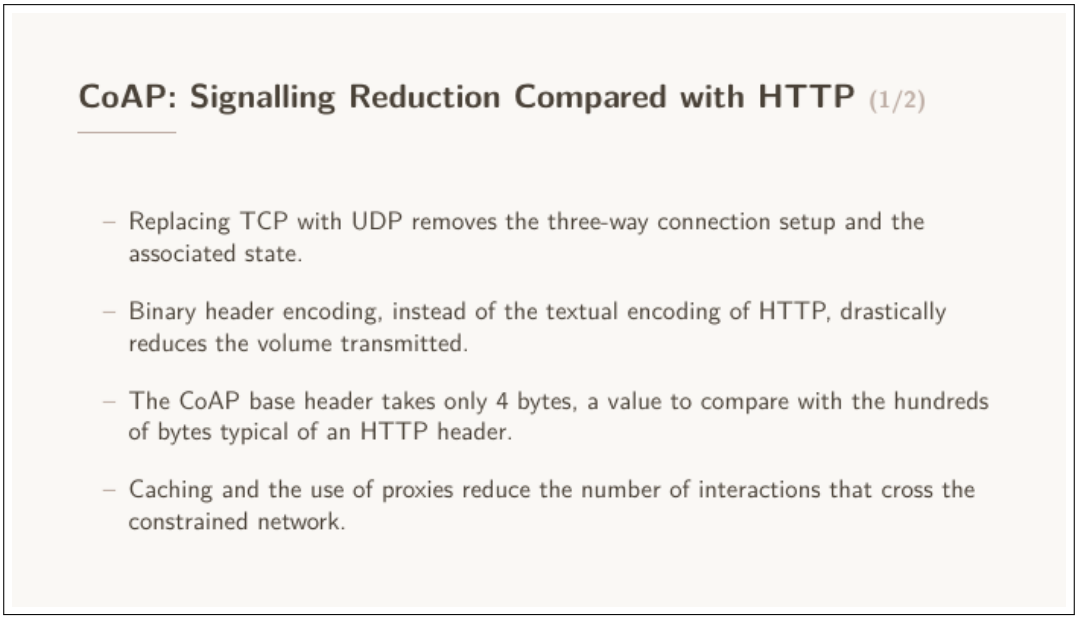
Choosing message by message. A reset message signals an error. Congestion control exists because UDP does not provide it. The flexibility to choose reliability per message is an advantage over TCP, which is all or nothing.

Slide 20. Confirmable and Non-Confirmable



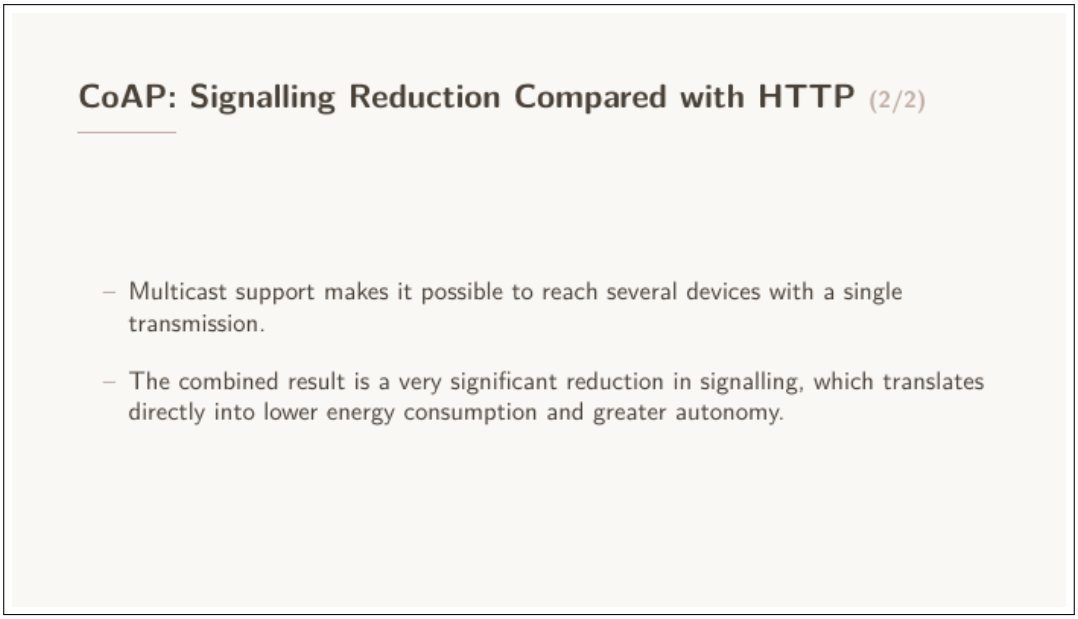
CON and NON in a picture. On the left, a confirmable message is lost, the sender waits, retransmits and receives the ACK. On the right, the non-confirmable message is sent with no expectation of reply. The choice depends on whether loss is tolerable.

Slide 21. CoAP: Signalling Reduction Compared with HTTP (1/2)



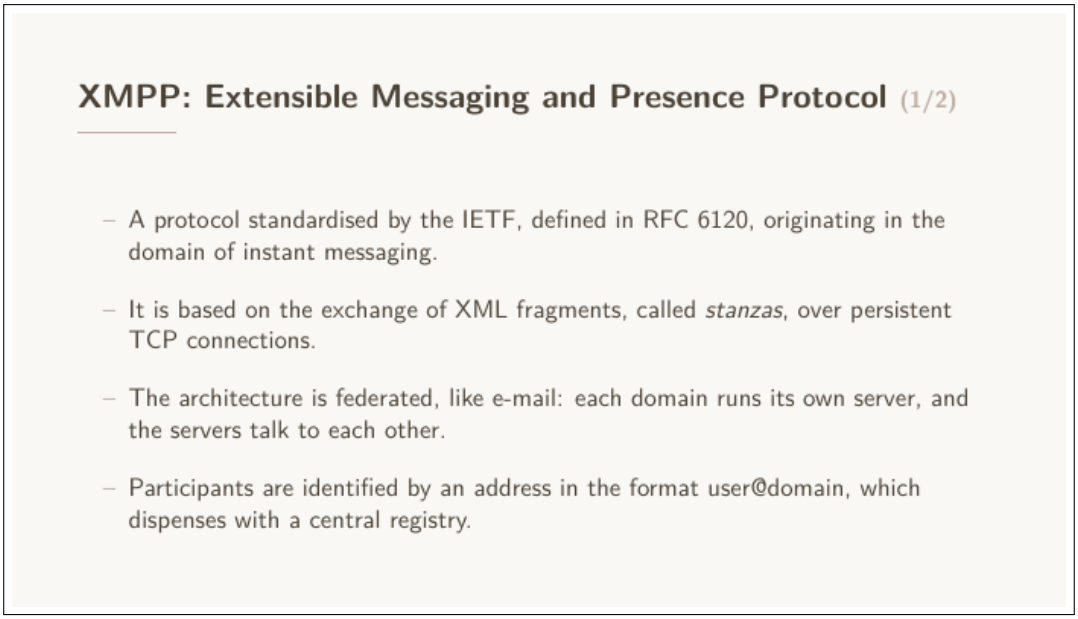
Where CoAP saves. With no TCP connection, no associated state and binary headers, the volume transmitted falls a lot. Caching and proxies further reduce the number of interactions that cross the constrained network.

Slide 22. CoAP: Signalling Reduction Compared with HTTP (2/2)



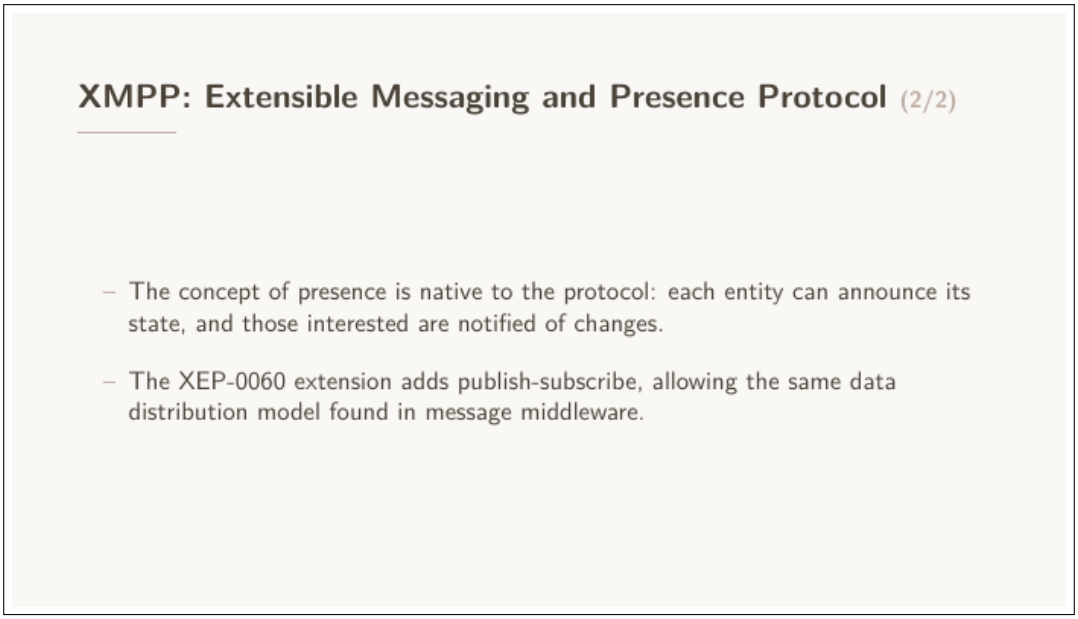
Less signalling, more autonomy. Multicast makes it possible to reach several devices at once. The combined effect is less signalling and, as a consequence, less energy consumed and greater autonomy.

Slide 23. XMPP: Extensible Messaging and Presence Protocol (1/2)



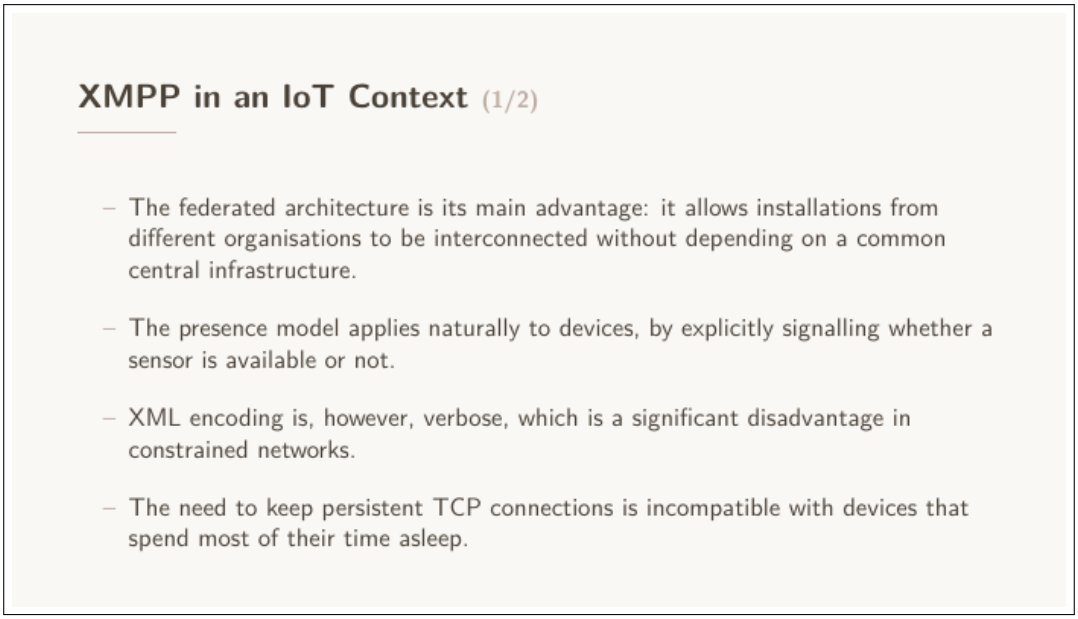
XMPP: messages and federation. XMPP comes from instant messaging. It exchanges XML fragments, the stanzas, over persistent TCP. The architecture is federated like e-mail, and participants have user@domain addresses.

Slide 24. XMPP: Extensible Messaging and Presence Protocol (2/2)



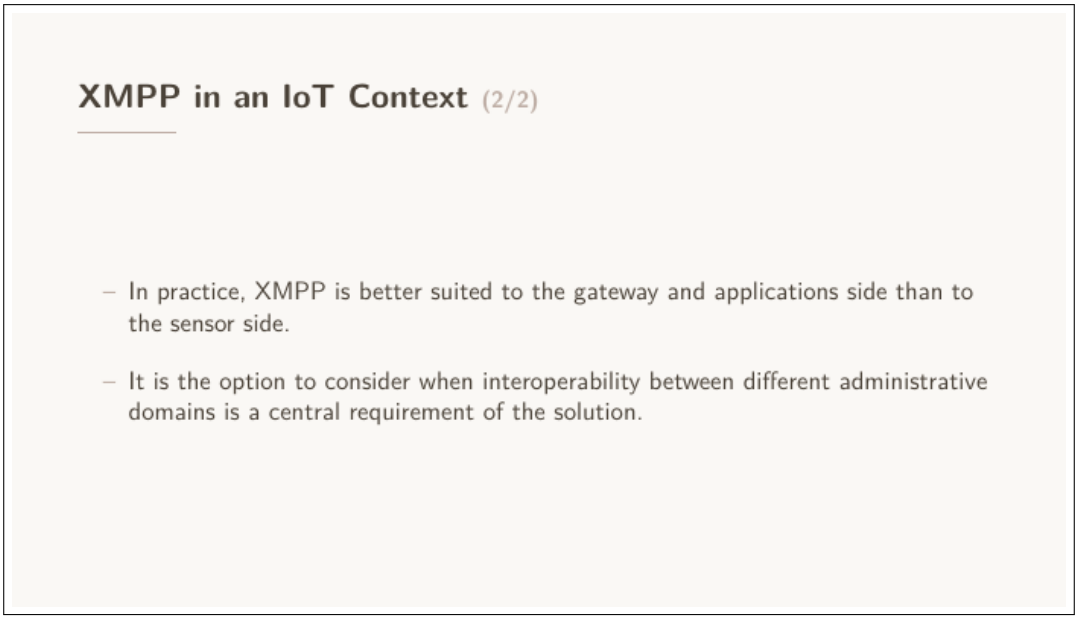
Presence and publish-subscribe. Presence is native: an entity announces its state and those interested are notified. The XEP-0060 extension adds publish-subscribe, which brings XMPP closer to the middleware of the next lecture.

Slide 25. XMPP in an IoT Context (1/2)



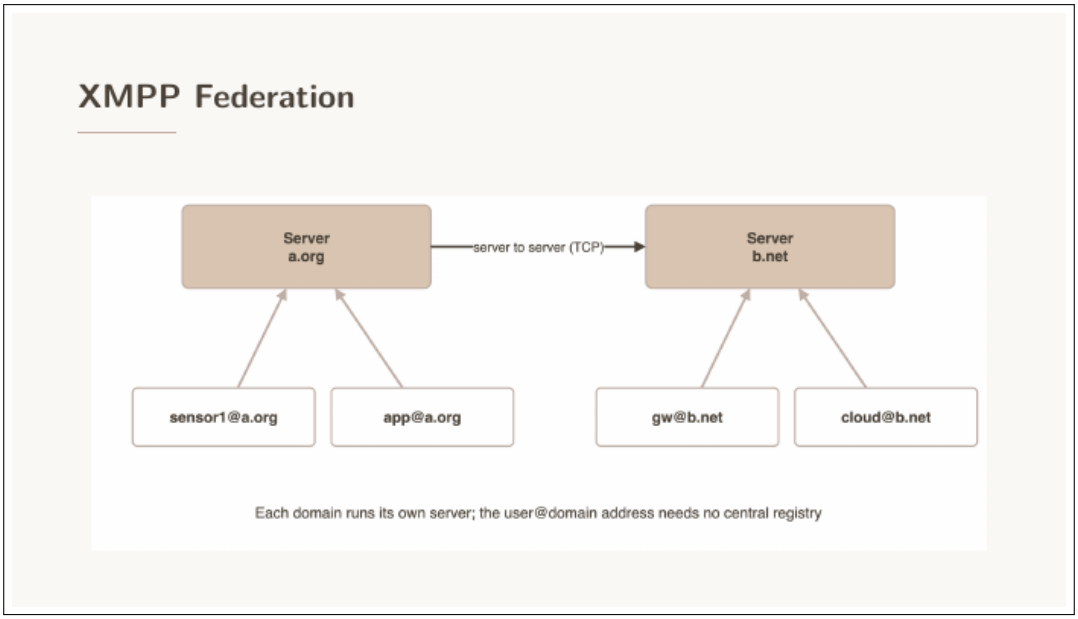
XMPP advantages in IoT. Federation allows organisations to be interconnected without a common central infrastructure, and presence describes well the availability of a device. The price is verbose XML and the persistent TCP connection.

Slide 26. XMPP in an IoT Context (2/2)



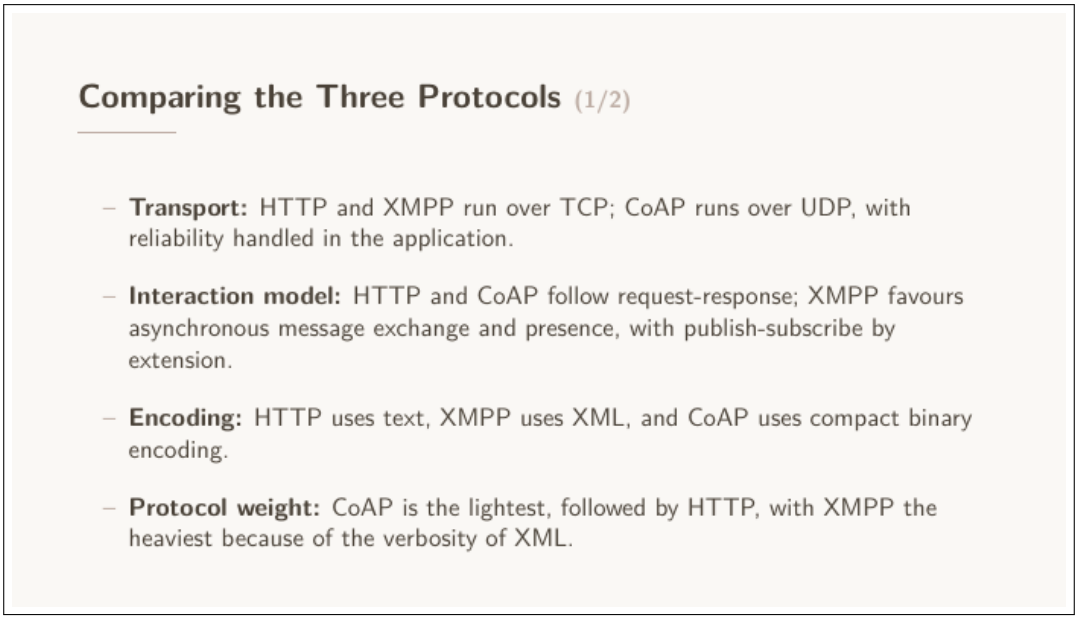
The gateway side. In practice, XMPP is better suited to the gateway and applications than to the sensor. It is the choice when interoperability between administrative domains is central.

Slide 27. XMPP Federation



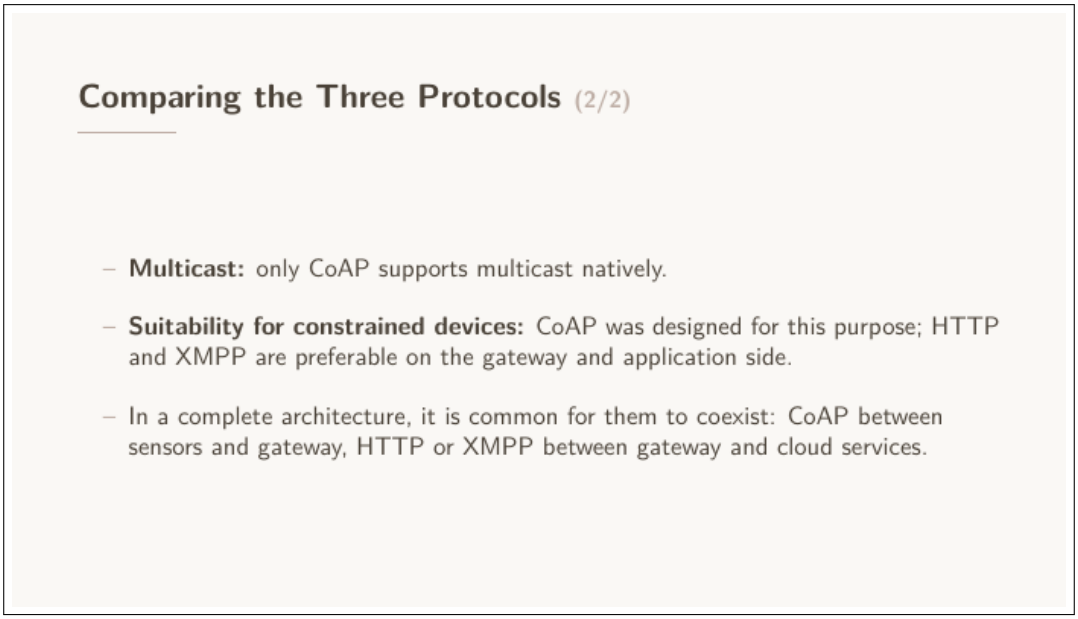
Federation in a picture. Two domains, each with its own server. Clients connect to their domain's server, and the servers talk to each other. The user@domain address needs no central registry.

Slide 28. Comparing the Three Protocols (1/2)



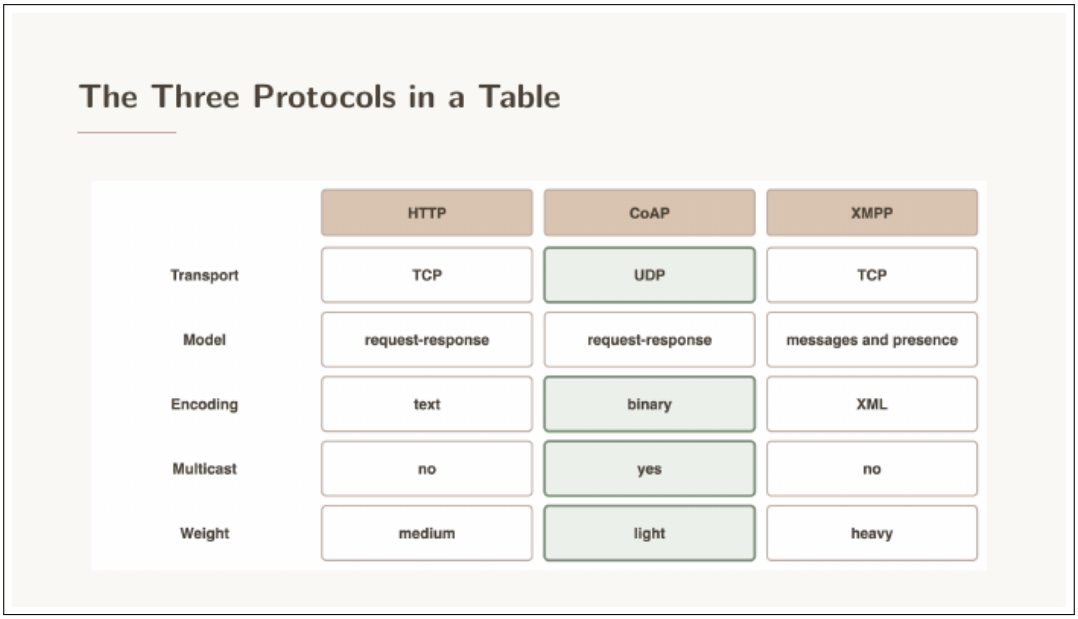
Four comparison criteria. Transport, interaction model, encoding and weight. CoAP is the only one over UDP and the only one with binary encoding. XMPP is the heaviest because of XML.

Slide 29. Comparing the Three Protocols (2/2)



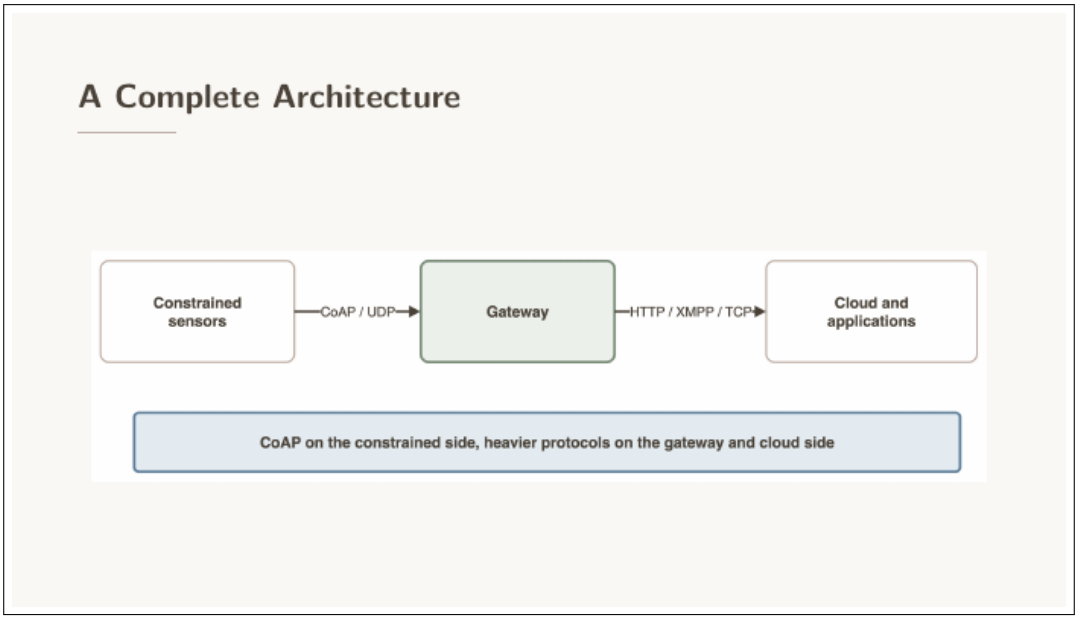
Multicast and suitability. Only CoAP has native multicast. For constrained devices, CoAP is the one designed for the purpose. In complete architectures, the three coexist in different zones.

Slide 30. The Three Protocols in a Table



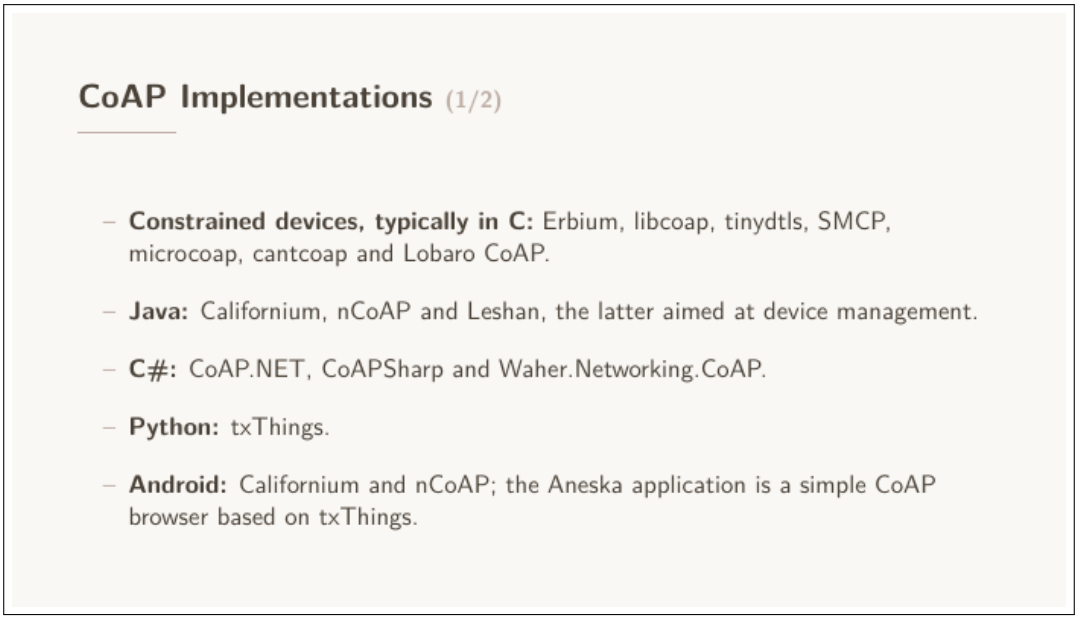
The comparison as a table. The table summarises the differences. You should be able to rebuild it: transport, model, encoding, multicast and weight, for each of the three protocols.

Slide 31. A Complete Architecture



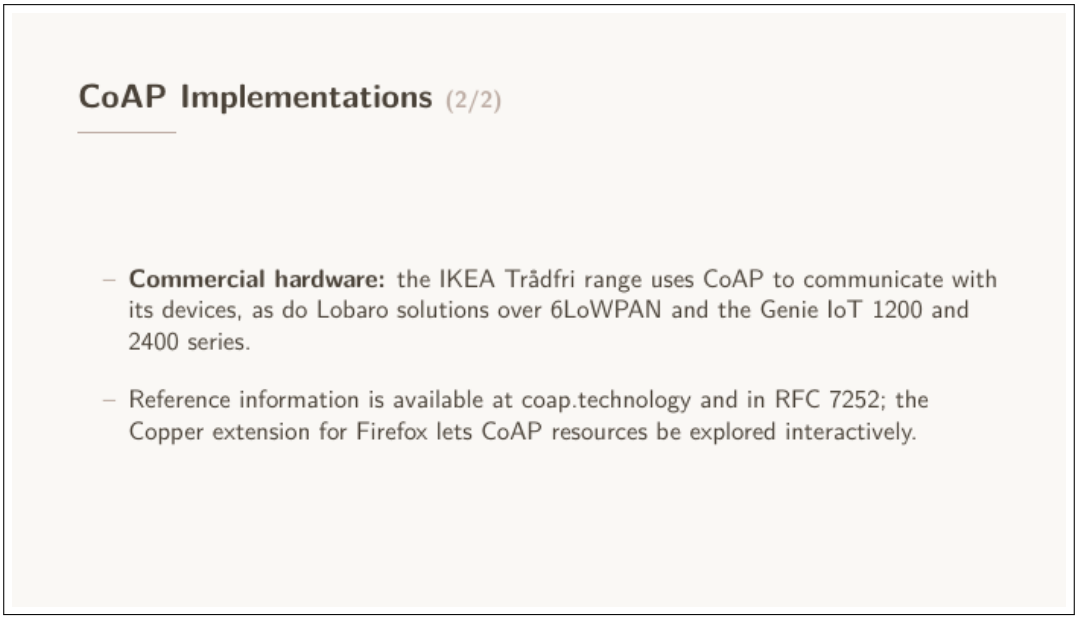
A complete architecture. CoAP on the constrained side, between sensors and gateway. HTTP or XMPP from the gateway to the cloud. Each protocol is used where its trade-off makes most sense.

Slide 32. CoAP Implementations (1/2)



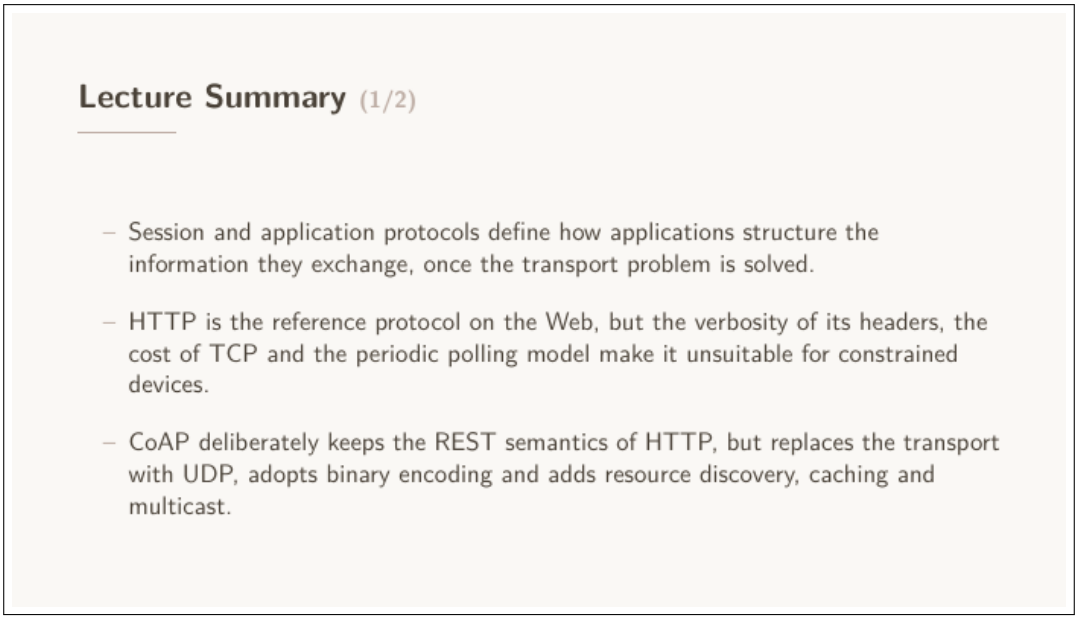
The CoAP ecosystem. There are implementations for constrained devices, Java, C#, Python and Android. You do not need to memorise names, but to know that CoAP has a mature ecosystem and what its axes are: device, server and tools.

Slide 33. CoAP Implementations (2/2)



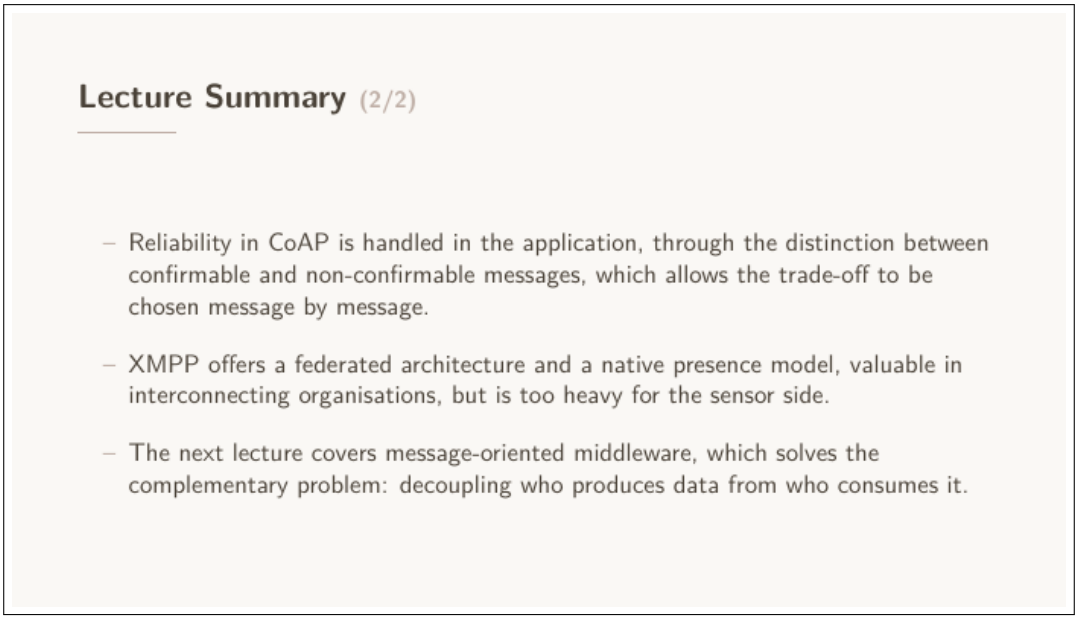
Hardware and references. CoAP is in commercial hardware, such as the IKEA Trådfri range. RFC 7252 and coap.technology are the references, and Copper lets resources be explored from the browser.

Slide 34. Lecture Summary (1/2)



The essentials of the lecture. HTTP serves the Web but weighs on constrained devices. CoAP keeps REST semantics and swaps the transport for UDP, with binary encoding, discovery, caching and multicast.

Slide 35. Lecture Summary (2/2)



What comes next. Reliability in CoAP is chosen per message, and XMPP belongs on the gateway side. The next lecture covers message-oriented middleware, which decouples who produces from who consumes.

Final Check

Before considering the material studied, check that you can answer the following without looking at the slides.

1. What messages are exchanged until an HTTP client receives a value from a sensor?
2. What are the fields of the CoAP base header and how large is it?
3. What is the difference between CON and NON messages and when is each used?
4. How does resource discovery work in CoAP?
5. Why is XMPP better suited to the gateway than to the sensor?
6. In what circumstances do CoAP, HTTP and XMPP coexist in the same architecture?

References

- Z. Shelby, K. Hartke, C. Bormann. *The Constrained Application Protocol (CoAP).* RFC 7252, 2014.
- Z. Shelby. *Constrained RESTful Environments (CoRE) Link Format.* RFC 6690, 2012.
- P. Saint-Andre. *Extensible Messaging and Presence Protocol (XMPP): Core.* RFC 6120, 2011.
- R. Fielding et al. *Hypertext Transfer Protocol (HTTP/1.1).* RFC 7230 to 7235, 2014.